App Dev 2 Project Report

1. Student Details

Name: Krish Gupta
Roll Number: 23f2000792
Email: 23f2000792@ds.study..iitm.ac.in
About Me: I am a student of the IIT Madras BS Degree Programme and I am interested in web development, data science and building practical software. I like working on projects where technology can replace repetitive manual work with a simple and usable system.

2. Project Details

Project Title: Placement Portal Application - V2

Problem Statement: Institutes handle placement activities involving students, companies and the placement team. When this work is managed through spreadsheets, emails and manual follow-ups, it becomes difficult to track company approvals, placement drives, applications and student status. The aim of this project is to provide one portal where these activities can be managed in an organised way.

Approach: I built the application as a role-based system with three users: Admin, Company and Student. Flask provides REST APIs and handles the database and business rules. Vue.js is used for the frontend. Redis is used for caching and as the Celery broker. Celery handles background work such as deadline reminders, monthly reports and CSV export. The frontend communicates with the backend through Axios.

3. AI/LLM Declaration

I used ChatGPT during development mainly for improving variable naming consistency and improving parts of the documentation. I also used it to discuss some code corrections when I faced integration issues. The approximate AI/LLM assistance in the overall project is around 10-15%. All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask + Flask-RESTful	Backend application and REST API endpoints
SQLAlchemy + SQLite	Database models, relationships and local persistence

Flask-Security-Too	Token authentication, password hashing and role control
Vue.js	Reactive frontend and page rendering
Vue Router + Pinia	Frontend routing and authentication state
Bootstrap 5 + Icons	Responsive layout, components and interface styling
Redis + Flask-Caching	Celery broker/result backend and dashboard caching
Celery + Celery Beat	Background jobs and scheduled tasks

5. Database Schema / ER Diagram

The Placement Portal Application uses a relational database to manage users, role-based access, student and company profiles, placement drives, and student applications. The principal database entities are User, Role, UserRoles, Student, Company, PlacementDrive, and Application.

Tables:

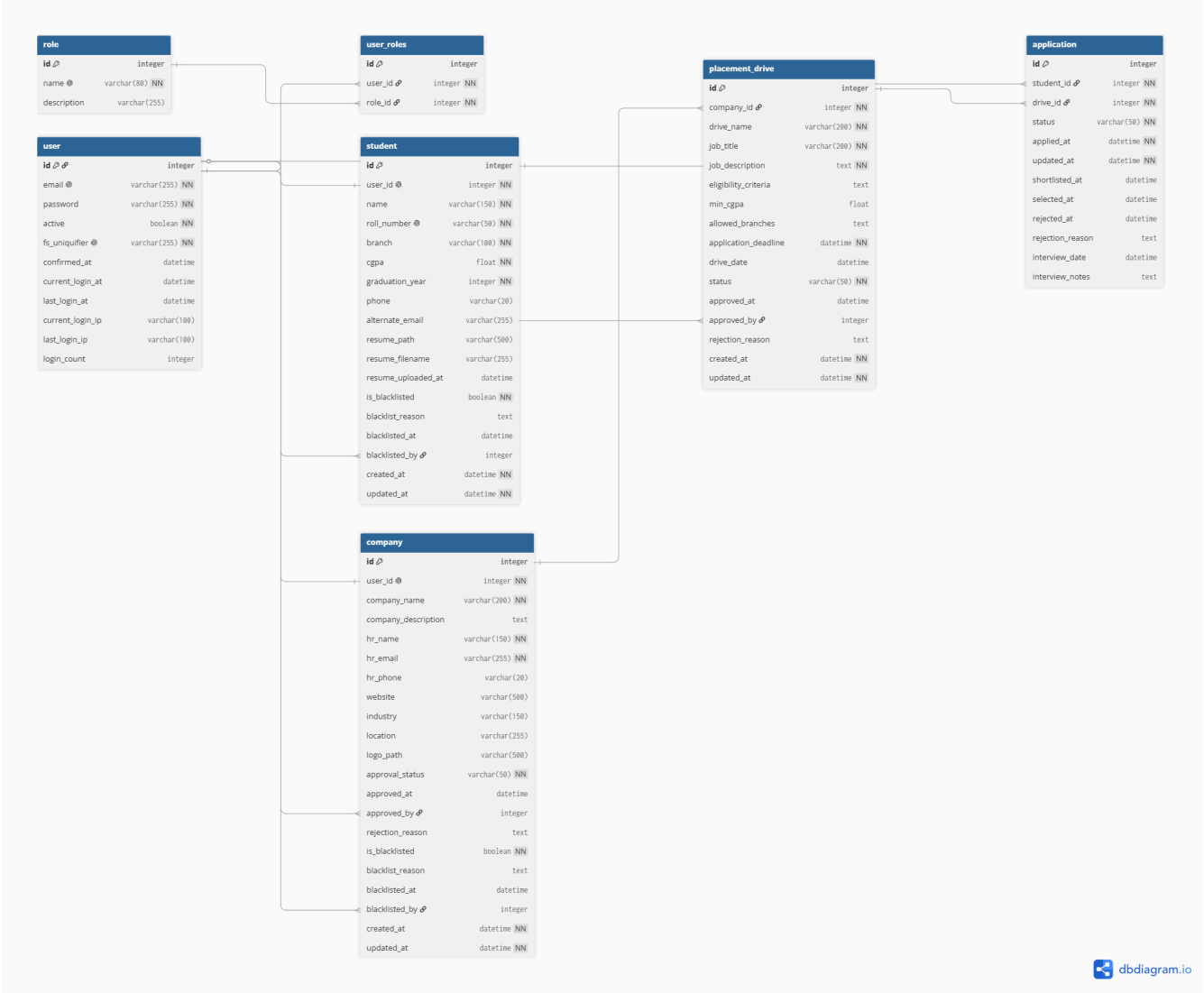
- **User** - Stores authentication and account details of all users, including administrators, students, and company representatives.
- **Role** - Stores predefined system roles such as Admin, Company, and Student.
- **UserRoles** - Association table that maps users to their assigned roles and supports role-based access control.
- **Student** - Stores student profile and placement-related information such as name, roll number, branch, CGPA, graduation year, contact details, resume information, and blacklist status.
- **Company** - Stores registered company details, HR contact information, industry and location details, approval status, and blacklist information.
- **PlacementDrive** - Stores placement drive details created by approved companies, including job title, job description, eligibility criteria, minimum CGPA, allowed branches, application deadline, drive date, and approval status.
- **Application** - Stores student applications submitted for placement drives. It tracks the application status, application date, interview schedule, shortlisting, rejection, and final selection details. A unique constraint prevents a student from applying to the same placement drive more than once.

Relationships:

- **Many-to-Many: User ↔ Role** through the UserRoles association table.
- **One-to-One: User → Student** for student accounts.
- **One-to-One: User → Company** for company accounts.
- **One-to-Many: Company → PlacementDrive** because a company can create multiple placement drives.
- **One-to-Many: Student → Application** because a student can submit applications to multiple placement drives.

- **One-to-Many: PlacementDrive → Application** because a placement drive can receive applications from multiple students.
- **Many-to-Many: Student ↔ PlacementDrive** indirectly through the Application table.

This schema supports the project's three-role architecture - Admin, Company, and Student - and the complete placement workflow from company registration and drive approval to student applications, interviews, and selection.



6. API Resource Endpoints

Endpoint	Method	Description
/api/login	POST	Authenticate a user
/api/admin/dashboard	GET	Get institute placement dashboard

/api/admin/companies/{id}/approve	POST	Approve a company
/api/student/drives	GET	List approved placement drives
/api/student/drives/{id}/apply	POST	Apply to a placement drive
/api/student/export/applications	POST	Start CSV export background job
/api/student/resume	POST / GET	Upload or download student resume
/api/company/drives	GET / POST	List or create company drives
/api/company/applications/{id}/shortlist	POST	Shortlist an applicant
/api/company/applications/{id}/select	POST	Mark a shortlisted student as selected

7. Architecture and Features (optional)

Architecture Overview:

- **backend/app.py –** creates the Flask application and registers REST API resources.
- **backend/controllers –** authentication, models, database setup and CRUD API logic.
- **backend/tasks.py –** Celery background and scheduled jobs.
- **backend/celery_worker.py –** Celery worker entry point.
- **frontend/src/views –** Admin, Company and Student pages.
- **frontend/src/services/api.js –** Axios API communication and authenticated file downloads.
- **frontend/src/stores –** Pinia stores for authentication and application messages.

Implemented Features:

- Role-based login for Admin, Company and Student.
- Admin approval, rejection, blacklisting and activation workflows.
- Company drive creation and admin approval before a drive becomes available.
- Student eligibility checks using CGPA and branch before application.
- Application tracking through applied, shortlisted, selected and rejected states.
- Interview scheduling by companies.
- PDF resume upload by students and authorised resume access for Admin and Companies.
- CSV export of student application history using a Celery background task.
- Daily deadline reminder and monthly placement report jobs using Celery Beat.
- Redis-backed caching for the admin dashboard.

8. Video Presentation

Drive Link:
https://drive.google.com/drive/folders/12gVcvUKRBx4KDL2yGwkZEWZEHYzt0Zey?usp=sharing